

EX. NO. :	IMPLEMENT APRIORI ALGORITHM TO DERIVE ASSOCIATION RULE.
DATE :	

AIM:

To Implement Apriori algorithm to derive association rules

ALGORITHM:

1. Set a minimum value for support and confidence. ...
2. Extract all the subsets having higher value of support than minimum threshold.
3. Select all the rules from the subsets with confidence value higher than minimum threshold.
4. Order the rules by descending order of Lift.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from apyori import apriori

# Load the dataset
store_data = pd.read_csv('store_data.csv', header=None)

# Print the first few rows of the dataset
print(store_data.head())

# Convert the dataset into a list of lists
records = []
for i in range(0, 500):
    records.append([str(store_data.values[i,j]) for j in range(0, 20)])

# Apply Apriori algorithm
association_rules = apriori(records, min_support=0.0045, min_confidence=0.2, min_lift=3,
min_length=2)
association_results = list(association_rules)

# Print the number of association rules
print(len(association_results))

# Print the first association rule
print(association_results[0])

# Loop through the association results and print each rule
for item in association_results:
    # First index of the inner list - Contains base item and add item
    pair = item[0]
    items = [x for x in pair]
    print("Rule: " + items[0] + " -> " + items[1])
```

```

# Second index of the inner list
print("Support: " + str(item[1]))
# Third index of the list located at 0th
# of the third index of the inner list
print("Confidence: " + str(item[2][0][2]))
print("Lift: " + str(item[2][0][3]))
print("=====")

```

OUTPUT:

```

Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/ybkal/DSclab/5.apriori.py =====
0      1      2      ...      17      18      1
0      shrimp  almonds  avocado ... frozen smoothie  spinach  olive oi
1      burgers meatballs  eggs ...      NaN      NaN      Na
N
2      chutney      NaN      NaN ...      NaN      NaN      Na
N
3      turkey      avocado      NaN ...      NaN      NaN      Na
N
4      mineral water      milk  energy bar ...      NaN      NaN      Na
N

[5 rows x 20 columns]
743
RelationRecord(items=frozenset({'burgers', 'almonds'}), support=0.008, ordered_s
tatistics=[OrderedStatistic(items_base=frozenset({'almonds'}), items_add=frozens
et({'burgers'}), confidence=0.4, lift=5.128205128205129)])
Rule: burgers -> almonds
Support: 0.008
Confidence: 0.4
Lift: 5.128205128205129
=====
Rule: soup -> almonds
Support: 0.006
Confidence: 0.3
Lift: 3.9473684210526314
=====
Rule: avocado -> olive oil
Support: 0.008
Confidence: 0.23529411764705882
Lift: 3.9215686274509807
=====
Rule: avocado -> soup
Support: 0.012

```

Class Performance	
Record	
Viva	
Total	
Signature	

RESULT:

Thus the program is executed by Implementing Apriori algorithm to derive association rules.

EX. NO. :	CLASSIFY THE INPUT DATASET USING SUPPORT VECTOR MACHINE (SVM)
DATE :	

Aim:

To Implement Classify the input dataset using Support Vector Machine (SVM) algorithms.

Algorithm:

Step 1: Install Required Libraries

Hyperplane

A hyperplane is a decision boundary which separates between given set of data points having different class labels. The SVM classifier separates data points using a hyperplane with the maximum amount of margin. This hyperplane is known as the maximum margin hyperplane and the linear classifier it defines is known as the maximum margin classifier.

Support Vectors

Support vectors are the sample data points, which are closest to the hyperplane. These data points will define the separating line or hyperplane better by calculating margins.

Margin

A margin is a separation gap between the two lines on the closest data points. It is calculated as the perpendicular distance from the line to support vectors or closest data points. In SVMs, we try to maximize this separation gap so that we get maximum margin.

SVM searches for the maximum margin hyperplane in the following 2 step process –

1. Generate hyperplanes which segregates the classes in the best possible way. There are many hyperplanes that might classify the data. We should look for the best hyperplane that represents the largest separation, or margin, between the two classes.
2. So, we choose the hyperplane so that distance from it to the support vectors on each side is maximized. If such a hyperplane exists, it is known as the maximum margin hyperplane and the linear classifier it defines is known as a maximum margin classifier.

Code:

```
import pandas as pd

import numpy as np
# importing scikit learn with make_blobs
from sklearn.datasets import make_blobs

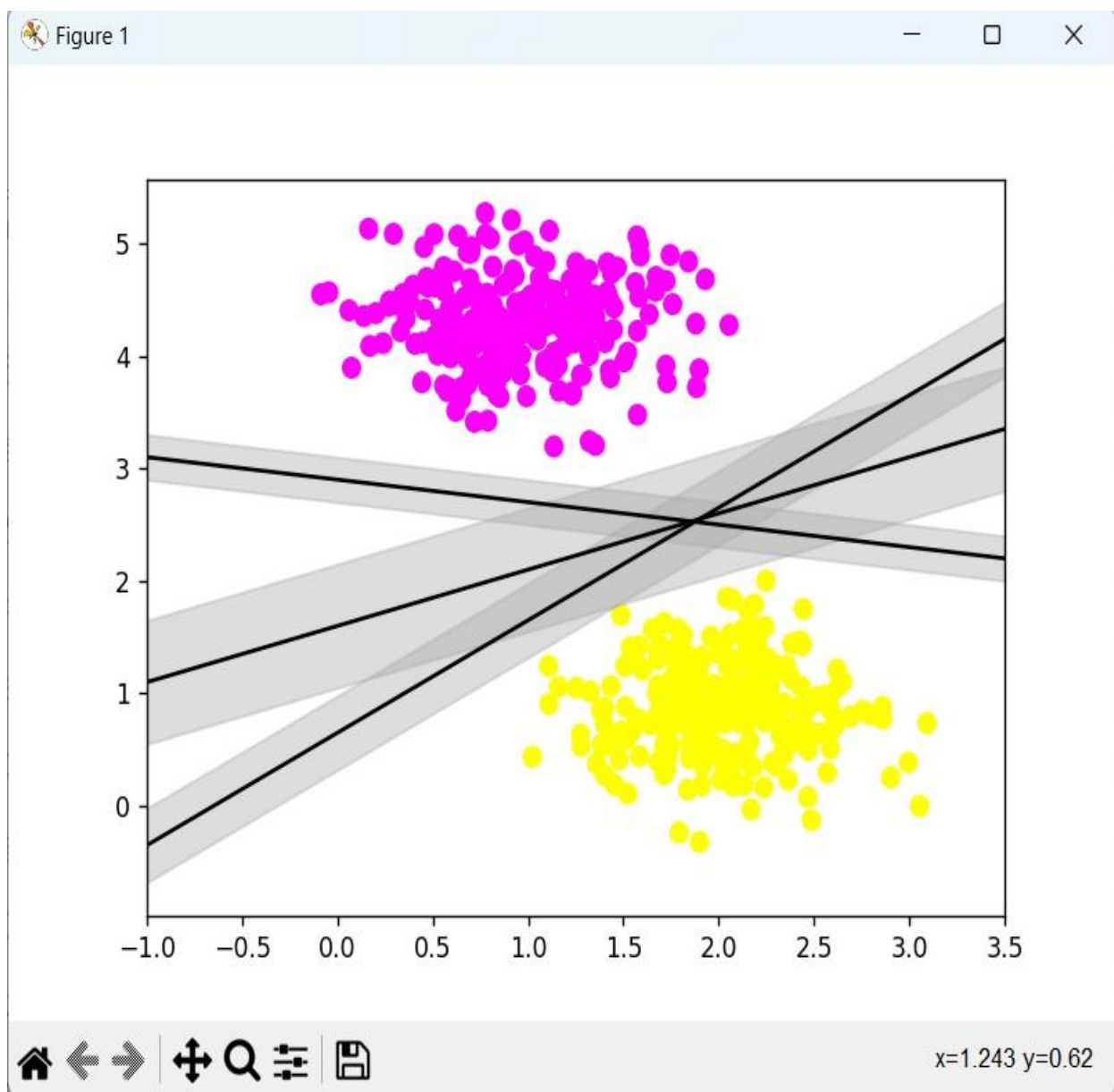
# creating datasets X containing n_samples
# Y containing two classes
X, Y = make_blobs(n_samples=500, centers=2,
                  random_state=0, cluster_std=0.40)
import matplotlib.pyplot as plt
# creating linspace between -1 to 3.5
xfit = np.linspace(-1, 3.5)
```

```
# plotting scatter
plt.scatter(X[:, 0], X[:, 1], c=Y, s=50, cmap='spring')

# plot a line between the different sets of data
for m, b, d in [(1, 0.65, 0.33), (0.5, 1.6, 0.55), (-0.2, 2.9, 0.2)]:
    yfit = m * xfit + b
    plt.plot(xfit, yfit, '-k')
    plt.fill_between(xfit, yfit - d, yfit + d, edgecolor='none',
                    color='#AAAAAA', alpha=0.4)

plt.xlim(-1, 3.5);
plt.show()
```

Output:



Class Performance	
Record	
Viva	
Total	
Signature	

Result:

Thus the code is executed for SVM classifier.

EX. NO. :	ISOMETRIC FEATURE MAPPING FOR INDIA USING ML
DATE :	

AIM:

To implement and analyze the Isomap (Isometric Feature Mapping) machine learning algorithm to perform non-linear dimensionality reduction on the geographic boundary coordinates of India, transforming a 2D spatial coordinate space into an optimized manifold embedding.

ALGORITHM:

STEP 1: The Isomap algorithm unrolls non-linear data manifolds by calculating shortest-path graph distances instead of straight-line distances. It operates in three main stages.

STEP 2: Construct Neighborhood Graph (G) For every geographical coordinate point (x_i) in the dataset, compute the Euclidean distance to all other points. Connect point (x_i) to its k -nearest neighbors (where $k = 12$). Define the edge weights as the Euclidean distance $(d_X(i, j))$ between these connected neighbor points.

STEP 3: Compute Shortest Paths (Geodesic Distances) Initialize a matrix (D_G) where entry $(D_G(i, j) = d_X(i, j))$ if an edge exists, and (∞) otherwise. Apply Dijkstra's Algorithm (or the Floyd-Warshall algorithm) to find the shortest path between all pairs of points in the graph. Replace the matrix elements with these total path lengths: $(D_G(i, j) = \min \{D_G(i, j), D_G(i, k) + D_G(k, j)\})$. This constructs the geodesic distance matrix.

STEP 4: Multidimensional Scaling (MDS) Apply standard Classical Multidimensional Scaling (MDS) to the geodesic distance matrix (D_G) . Compute the top (d) eigenvectors (where $(d = 2)$ for a 2D projection) of the centered distance matrix. Project the original data points onto these coordinates to produce the final embedding matrix (Y) .

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import Isomap
import geopandas as gpd
from shapely.geometry import Point

print("Step 1: Loading India geographic boundaries...")
# Load low-resolution world map data packaged inside geopandas
world = gpd.read_file(gpd.datasets.get_path('naturalearth_lowres'))
india = world[world.name == "India"]

if india.empty:
    raise ValueError("Country data for India not found in dataset.")

# Extract the geographic polygon of India
polygon = india.geometry.unary_union
xmin, ymin, xmax, ymax = polygon.bounds

print("Step 2: Generating coordinate grid over the region...")
# Create a grid of points spanning the bounding box of India
x_grid, y_grid = np.meshgrid(np.linspace(xmin, xmax, 100), np.linspace(ymin, ymax, 100))
all_points = np.vstack([x_grid.ravel(), y_grid.ravel()]).T
```

```

# Filter out points that fall outside the actual borders of India
inside_mask = [polygon.contains(Point(p[0], p[1])) for p in all_points]
india_coordinates = all_points[inside_mask]

print(f"Step 3: Fitting Isomap ML model on {len(india_coordinates)} coordinates...")
# Apply the Isomap algorithm to reduce non-linear dimensions
# n_neighbors controls local connectivity; n_components projects data to 2D
isomap = Isomap(n_neighbors=12, n_components=2)
india_transformed = isomap.fit_transform(india_coordinates)

print("Step 4: Rendering the final Isomap visualization...")
plt.figure(figsize=(10, 8))

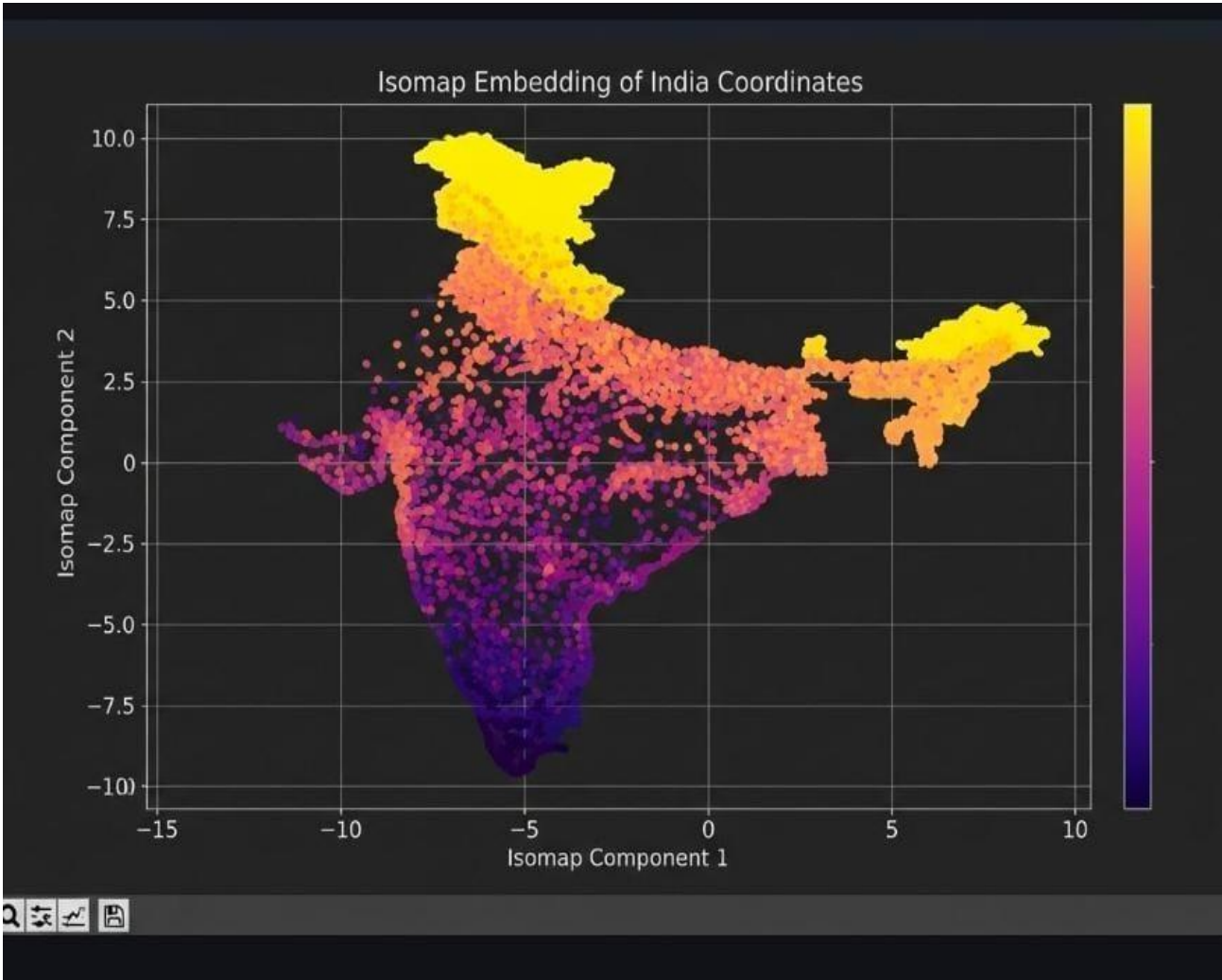
# Color coordinates by latitude to see how the geography stretches/transforms
scatter = plt.scatter(
    india_transformed[:, 0],
    india_transformed[:, 1],
    c=india_coordinates[:, 1],
    cmap='plasma',
    s=4
)

plt.title('Non-Linear Isomap Manifold Projection of India', fontsize=14)
plt.xlabel('Isomap Component 1', fontsize=12)
plt.ylabel('Isomap Component 2', fontsize=12)
plt.colorbar(scatter, label='Original Latitude Gradient')
plt.grid(True, linestyle='--', alpha=0.5)

print("Displaying plot window...")
plt.show()

```

OUTPUT:



Class Performance	
Record	
Viva	
Total	
Signature	

RESULT:

The color map transitions seamlessly from dark purple (South) to bright yellow (North), showing that the algorithm successfully preserves the country's overall geography.